UNIVERSIDAD TECNOLÓGICA DEL VALLE DE TOLUCA

DIRECCIÓN DE CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN Y COMUNICACIÓN

ASIGNATURA:

Fundamentos de Inteligencia Artificial

DOCENTE:

Hector Velazquez Estrada

Manual Técnico

PRESENTA:

222410897 - Gutierrez Ugalde Samuel
222410935 - Nava Garcia Rodrigo
222411182 - Castillo Ramirez Nahum Yabet
222410915 - Ramirez Romero Pablo
222410899 - Peña Hernandez Alexander

GENERACIÓN:

Septiembre 2024 - Agosto 2026

Santa María Atarasquillo, Lerma, Estado de México. 21 de agosto de 2026

MANUAL TÉCNICO OFICIAL: MEDREMINDER AI

Versión del Sistema: 1.0.0 (Proof of Concept - PoC)

Clasificación: Documentación de Arquitectura e Ingeniería de Software

ÍNDICE GENERAL

1. **1. INTRODUCCIÓN Y ALCANCE DEL PROTOTIPO**
    - 1.1 1.1 Contextualización y Propósito
    - 1.2 1.2 Alcance Funcional y Límites del Prototipo
2. **2. REQUISITOS DE ENTORNO E INFRAESTRUCTURA**
    - 2.1 2.1 Requisitos de Software y Runtime
    - 2.2 2.2 Eficiencia de Despliegue y Cero Dependencias Exógenas
3. **3. ARQUITECTURA DEL SISTEMA: ESTADO VOLÁTIL EN MEMORIA**
    - 3.1 3.1 Patrón Arquitectónico de Estado Volátil
    - 3.2 3.2 Desacoplamiento Vista-Controlador (MVC Simulado)
    - 3.3 3.3 Rendimiento y Latencia Sub-Milisegundo
4. **4. DESCRIPCIÓN DE COMPONENTES DE INTERFAZ DE USUARIO (UI/UX)**
    - 4.1 4.1 Formulario de Captura de Datos
    - 4.2 4.2 Tabla Dinámica de Registros (ttk.Treeview)
    - 4.3 4.3 Módulo de Alertas y Manejo de Excepciones
5. **5. LÓGICA DE OPERACIONES CRUD (CREATE, READ, UPDATE, DELETE)**
    - 5.1 5.1 Operación de Creación (Create / Insertion)
    - 5.2 5.2 Operación de Lectura y Visualización (Read / Query)
    - 5.3 5.3 Operación de Actualización (Update / Modification)
    - 5.4 5.4 Operación de Eliminación (Delete / Removal)
6. **6. MOTOR FORMAL DE LÓGICA PROPOSICIONAL Y MODELADO MATEMÁTICO**
    - 6.1 6.1 Formalización Lógica de las Reglas de Validación de Entrada (CRUD)
    - 6.2 6.2 Regla Lógica para el Disparo de Alerta por Inventario Crítico
    - 6.3 6.3 Modelado de Transición de Estados y Exclusión Mutua de Descuento

INTRODUCCIÓN

En el contexto de la ingeniería de software aplicada a la salud mental y al seguimiento farmacológico, la precisión de los datos y la inmediatez de la interfaz determinan la adherencia terapéutica del usuario. MedReminder AI surge como una iniciativa académica de alto nivel dentro de la Universidad Tecnológica del Valle de Toluca, en la Dirección de Carrera de Tecnologías de la Información y Comunicación, para dar respuesta a la imperiosa necesidad de contar con esquemas de notificación médica predictivos y formalmente validados.

El desarrollo de esta Prueba de Concepto (PoC) responde al rigor metodológico exigido por la asignatura de Fundamentos de Inteligencia Artificial. Su objetivo primordial es establecer un laboratorio de software donde la lógica de proposiciones, la teoría de autómatas y los patrones de diseño orientados a eventos converjan en un entorno cliente de escritorio caracterizado por cero latencia, total determinismo y una arquitectura inmune a fallas de persistencia diferida.

Mediante este Manual Técnico Oficial, se documenta no solo la estructura de componentes e interfaces de usuario, sino también el motor matemático que sostiene cada decisión de validación, confirmación y descuento de inventario. Este enfoque garantiza una base conceptual sólida para futuras iteraciones del sistema orientadas a despliegues distribuidos en la nube y la integración de modelos generativos de asistencia al paciente.

1. INTRODUCCIÓN Y ALCANCE DEL PROTOTIPO

1.1 Contextualización y Propósito

El presente documento constituye el Manual Técnico Oficial del sistema MedReminder AI, una solución tecnológica concebida para la gestión, seguimiento y notificación de esquemas y recordatorios médicos. El proyecto se constituye formalmente como una Prueba de Concepto en fase de prototipado de escritorio, diseñada específicamente para evaluar la usabilidad, la fluidez de interacción del usuario (UX) y el comportamiento dinámico de los componentes en tiempo de ejecución.

Objetivos generales integrados:

1. **Minimizar deficiencias y errores médicos:** Reducir drásticamente los riesgos asociados con el olvido de tomas, la confusión en las dosis prescritas y la falta de seguimiento de inventario.
2. **Garantizar el cumplimiento riguroso de tratamientos farmacológicos:** Asegurar la adherencia al tratamiento en pacientes con enfermedades crónicas o polimedicación mediante recordatorios inteligentes y reglas formales de lógica.
3. **Automatizar el control de inventarios de medicamentos:** Proveer un sistema de supervisión y conteo en tiempo real para alertar de forma oportuna sobre stocks críticos y prevenir desabastecimientos.
4. **Facilitar la toma de decisiones asistida:** Integrar un motor de razonamiento lógico transparente que evalúe y valide la seguridad en la administración de medicamentos.

1.2 Alcance Funcional y Límites del Prototipo

El alcance de esta versión PoC abarca los siguientes aspectos operativos:

- La validación de patrones de entrada de datos para regímenes farmacos (Nombre del Medicamento, Dosis Prescrita, Cantidad/Frecuencia).
- La evaluación de tiempo de respuesta de la interfaz gráfica de usuario en entornos cliente local.
- La verificación de los flujos de alerta activa y diálogos interactivos de seguridad operativa.

Nota de Arquitectura: Como vehículo de pruebas, el prototipo opera de manera aislada en el espacio de usuario del sistema operativo cliente, abstrayendo la complejidad de infraestructura de persistencia remota para priorizar la agilidad funcional y la retroalimentación inmediata de la interfaz.

2. REQUISITOS DE ENTORNO E INFRAESTRUCTURA

2.1 Requisitos de Software y Runtime

Para la ejecución transparente de MedReminder AI, el entorno host requiere únicamente de los siguientes componentes estándar:

Componente	Especificación Requerida	Observaciones de Ingeniería
Entorno de Ejecución	Python 3.8 o superior	Se recomienda arquitectura 64-bit para gestión óptima de memoria.
Gestor de Ventanas	Tcl/Tk nativo	Integrado por defecto en el paquete estándar de Python.
Sistema Operativo	Multiplataforma	Compatible con Windows 10/11, GNU/Linux (Kernel 5.x+) y macOS (11.0+).

2.2 Eficiencia de Despliegue y Cero Dependencias Exógenas

Una de las ventajas de la arquitectura adoptada radica en el Principio de Dependencia Cero. El sistema aprovecha íntegramente los paquetes nativos de la biblioteca estándar de Python (tkinter, tkinter.ttk, tkinter.messagebox). Esto deriva en beneficios clave:

1. **Despliegue Inmediato (Zero-Install Overhead):** Elimina la necesidad de gestores de paquetes externos como pip o entornos virtuales complejos (venv/conda).

2. **Ausencia de Sobrecarga de Base de Datos:** Al no integrar motores de bases de datos relacionales (RDBMS) ni no relacionales pesados, el footprint en disco y en memoria es mínimo, reduciendo vectores de fallo por configuraciones de conectores DB o esquemas de migración.

3. ARQUITECTURA DEL SISTEMA: ESTADO VOLÁTIL EN MEMORIA

3.1 Patrón Arquitectónico de Estado Volátil

El núcleo conceptual de MedReminder AI implementa una arquitectura basada en **Estado Volátil en Memoria RAM**. Los datos del dominio operativo no se persisten en almacenamiento secundario (disco duro/SSD), sino que residen en la pila de memoria del proceso en ejecución.

La fuente de la verdad del prototipo consiste en un contenedor de tipo list de Python que alberga elementos del tipo dict (diccionarios). Cada diccionario representa una entidad Medicamento con llaves tipadas:

- 'nombre' (String): Denominación comercial o genérica del fármaco.
- 'dosis' (String/Float): Concentración o posología requerida.
- 'cantidad' (Integer/String): Unidades disponibles o frecuencia de administración.

3.2 Desacoplamiento Vista-Controlador (MVC Simulado)

La arquitectura establece un desacoplamiento riguroso entre la capa de presentación y la capa de lógica de negocio:

- **Capa de Vista (View):** Construida sobre componentes ttk.Treeview y controles contenedores (ttk.Frame), encargada exclusivamente de la renderización visual y la captura de eventos de periféricos (mouse/teclado).

- **Capa del Controlador/Lógica (Controller/Model):** Rutinas en Python puro que gestionan mutaciones sobre la lista en RAM y notifican a la vista para forzar el re-pintado del árbol de elementos.

3.3 Rendimiento y Latencia Sub-Milisegundo

Al prescindir de llamadas a disco o sockets TCP/IP hacia servidores de bases de datos remotos, las operaciones de lectura y modificación se realizan a velocidad de bus de memoria interna (RAM), garantizando una latencia de respuesta sub-milisegundo y eliminando por completo cuellos de botella por competencia de hilos o bloqueos de transacciones (*locks*).

4. DESCRIPCIÓN DE COMPONENTES DE INTERFAZ DE USUARIO (UI/UX)

4.1 Formulario de Captura de Datos

El módulo de entrada está compuesto por controles agrupados dentro de un contenedor estructurado (ttk.LabelFrame):

- **Campos de Entrada (ttk.Entry):** Capturan dinámicamente los atributos de los medicamentos.
- **Etiquetas Identificativas (ttk.Label):** Guían la interacción y proveen accesibilidad básica.

4.2 Tabla Dinámica de Registros (ttk.Treeview)

La visualización tabular utiliza el widget ttk.Treeview configurado en modo multicolumna:

- **Columnas Tabulares:** Identificadores mapeados para *Nombre*, *Dosis* y *Cantidad*.
- **Binding de Selección:** Permite la selección interactiva de filas mediante eventos de clic, permitiendo mapear el índice visual seleccionado con el índice correspondiente en la lista en memoria RAM.

4.3 Módulo de Alertas y Manejo de Excepciones

La prevención de fallos de entrada y la robustez del prototipo están aseguradas mediante el submódulo tkinter.messagebox:

- **Validación de Campos Vacíos:** Comprobación condicional preventiva previa a la inserción. Si un campo carece de caracteres significativos (aplicación de .strip()), se dispara una alerta de advertencia (messagebox.showwarning).

- **Confirmaciones Operativas:** Diálogos modales de confirmación antes de ejecutar operaciones destructivas (como la eliminación de un registro).

5. LÓGICA DE OPERACIONES CRUD (CREATE, READ, UPDATE, DELETE)

5.1 Operación de Creación (Create / Insertion)

1. **Captura:** Recuperación de los valores textuales desde los widgets Entry.
2. **Sanitización y Validación:** Verificación de que ningún campo esté en blanco.
3. **Instanciación:** Creación de un nuevo objeto diccionario: nuevo_registro = {'nombre': n, 'dosis': d, 'cantidad': c}.
4. **Append en Memory Store:** Adición del diccionario a la lista global mediante .append().

5.2 Operación de Lectura y Visualización (Read / Query)

1. **Poblado Inicial / Refresco:** Limpieza del contenedor Treeview mediante la eliminación de todos los nodos (tree.delete(*tree.get_children())).
2. **Iteración Vectorial:** Recorrido en bucle for sobre la lista de diccionarios en RAM.
3. **Inyección en Vista:** Inserción de cada elemento mediante tree.insert('', 'end', values=(...)).

5.3 Operación de Actualización (Update / Modification)

1. **Identificación de Índice:** Obtención del ítem seleccionado en el Treeview mediante la función tree.selection().
2. **Mutación en RAM:** Sustitución de los valores en el diccionario almacenado en el índice correspondiente dentro de la lista global.
3. **Rediseño:** Actualización directa del nodo visual mediante la reasignación de tuplas de valores en el Treeview.

5.4 Operación de Eliminación (Delete / Removal)

6. MOTOR FORMAL DE LÓGICA PROPOSICIONAL Y MODELADO MATEMÁTICO

6.1 Formalización Lógica de las Reglas de Validación de Entrada (CRUD)

Para garantizar la integridad estricta de las entidades en la capa de datos en memoria RAM, el módulo de captura evalúa la validez semántica y sintáctica de

cada registro mediante una función proposicional bicondicional. Sean las variables proposicionales primitivas definidas sobre los campos de entrada:

- p: El atributo 'Nombre del Medicamento' no está vacío y satisface len(nombre.strip()) > 0.
- q: El atributo 'Dosis Prescrita' satisface len(dosis.strip()) > 0.
- r: El atributo 'Cantidad/Frecuencia' satisface len(cantidad.strip()) > 0.

La validez total del registro (V) se modela mediante la conjunción lógica triple:

$V \leftrightarrow (p \land q \land r)$

Por aplicación directa de las Leyes de De Morgan y la negación de proposiciones compuestas, el estado de fallo e inserción inválida (¬V), que gatilla la excepción modal via tkinter.messagebox.showwarning, responde formalmente a:

$\neg V \leftrightarrow \neg(p \land q \land r) \leftrightarrow (\neg p \lor \neg q \lor \neg r)$

Tabla de verdad del predicado de validación:

p	q	r	V = p ∧ q ∧ r	Acción del Sistema
V	V	V	V (Verdadero)	Persistencia en RAM / Append
V	V	F	F (Falso)	Disparo de Alerta (¬r)
V	F	V	F (Falso)	Disparo de Alerta (¬q)
V	F	F	F (Falso)	Disparo de Alerta (¬q ∨ ¬r)
F	V	V	F (Falso)	Disparo de Alerta (¬p)
F	V	F	F (Falso)	Disparo de Alerta (¬p ∨ ¬r)
F	F	V	F (Falso)	Disparo de Alerta (¬p ∨ ¬q)
F	F	F	F (Falso)	Disparo de Alerta (Bloqueo Total)

6.2 Regla Lógica para el Disparo de Alerta por Inventario Crítico

El motor de inferencia en tiempo real evalúa la necesidad de notificación de reposición de stock (Alerta A) mediante un sistema deductivo basado en el estado activo del fármaco y el umbral de unidades. Sean las variables:

- a: El tratamiento correspondiente se encuentra en estado Activo (estado == 'Activo').
- m: El volumen de unidades remanentes es menor o igual al umbral mínimo de seguridad (stock <= umbral_minimo).

- c: Existencia de orden de compra o reposición en curso (orden_compra == True).

El predicado proposicional de alerta de reposición (A) queda definido formalmente por:

$A \leftrightarrow (a \land m \land \neg c)$

Tabla de verdad del motor de alertas de inventario:

a	m	c	A = a ∧ m ∧ ¬c	Estado de Notificación
V	V	F	V (Verdadero)	Emisión Prioritaria de Alerta UI
V	V	V	F (Falso)	Silenciado (Reposición en Tránsito)
V	F	F	F (Falso)	Stock Nominal Suficiente
F	V	F	F (Falso)	Tratamiento Inactivo / Ignorado

6.3 Modelado de Transición de Estados y Exclusión Mutua de Descuento

El procesamiento de eventos de confirmación de toma posológica implementa una función de transición de estados determinista con mutación disjunta de inventario. Sea la variable proposicional T la confirmación afirmativa del usuario ("Dosis Tomada") y D la ejecución de la función decrementadora del stock en RAM (stock = stock - 1). La relación biunívoca establece:

$D \leftrightarrow T$

Para prevenir inconsistencias de doble descuento o saltos de secuencia posológica, las transiciones hacia los estados finales 'Omitido' (O) o 'Pospuesto' (P) imponen exclusividad mutua absoluta mediante el operador O Exclusivo (XOR - ⊕):

$Estado_Final = T \oplus O \oplus P$

Garantizando formalmente que $(O \vee P) \rightarrow \neg D$, conservando la invariante del volumen numérico de stock.

6.4 Especificación Técnica e Implementación de Referencia en Python

A continuación se expone la implementación de referencia en código fuente Python del motor formal de evaluación lógica, integrando tipado estático estricto, evaluación booleana pura y documentación embebida de tablas de verdad:

1.

```
from typing import Dict, Any, Tuple

def evaluar_validacion_crud(nombre: str, dosis: str, cantidad: str) -> Tuple[bool, str]:
    """
    Evalúa la validez formal del registro usando la proposición: V <-> (p ^ q ^ r)
    Aplicando Leyes de De Morgan para el diagnóstico de error: ¬V <-> (¬p v ¬q v ¬r)

    Tabla de Verdad Embebida:
    +---+---+---+----------------+----------------------+
    | p | q | r | V = p ^ q ^ r  | Estado de Salida     |
    +---+---+---+----------------+----------------------+
    | 1 | 1 | 1 | 1 (Verdadero)  | Registro Válido (RAM) |
    | 0 | X | X | 0 (Falso)      | Error en Prop (p)    |
    | X | 0 | X | 0 (Falso)      | Error en Prop (q)    |
    | X | X | 0 | 0 (Falso)      | Error en Prop (r)    |
    +---+---+---+----------------+----------------------+
    """
    p: bool = len(nombre.strip()) > 0
    q: bool = len(dosis.strip()) > 0
    r: bool = len(cantidad.strip()) > 0

    valido: bool = p and q and r

    if valido:
        return True, "Validación exitosa: Registro conforme."

    # Diagnóstico deductivo por De Morgan
    errores = []
    if not p: errores.append("Atributo 'Nombre' vacuo (¬p)")
    if not q: errores.append("Atributo 'Dosis' vacuo (¬q)")
    if not r: errores.append("Atributo 'Cantidad' vacuo (¬r)")

    return False, f"Fallo de Validación (¬V): {', '.join(errores)}"


def evaluar_alerta_inventario(activo: bool, stock: int, umbral: int, orden_en_curso:
bool) -> bool:
    """
    Calcula el predicado de alerta de reposición: A <-> (a ^ m ^ ¬c)
```

"""
a: bool = activo
m: bool = stock <= umbral
c: bool = orden_en_curso

alerta_activa: bool = a and m and (not c)
return alerta_activa

def procesar_transicion_estado(fármaco: Dict[str, Any], evento: str) -> Dict[str, Any]:
"""
Modela la transición de estados y la regla D <-> T respetando exclusividad mutua.
Eventos válidos: 'TOMAR', 'OMITIR', 'POSPONER'
"""
t: bool = (evento == 'TOMAR')
o: bool = (evento == 'OMITIR')
p: bool = (evento == 'POSPONER')

Invariante formal: Exclusividad mutua XOR (t ⊕ o ⊕ p)
if sum([t, o, p]) != 1:
raise ValueError("Violación de exclusividad mutua en transición de estados.")

if t:
Aplicación de D <-> T (Descuento explícito de inventario)
fármaco['stock'] = max(0, fármaco.get('stock', 0) - 1)
fármaco['estado_ultimo_recordatorio'] = 'Atendido'
elif o:
fármaco['estado_ultimo_recordatorio'] = 'Omitido'
elif p:
fármaco['estado_ultimo_recordatorio'] = 'Pospuesto'

return fármaco

2. **Verificación de Selección:** Confirmación de que el usuario ha marcado una fila válida.
3. **Confirmación Modal:** Interrupción por diálogo de confirmación (messagebox.askyesno).
4. **Extracción por Índice:** Eliminación del diccionario de la lista en RAM utilizando la instrucción del lista[indice] o .pop(indice).
5. **Actualización Visual:** Remoción del nodo visual mediante tree.delete(item_seleccionado).

6. CONCLUSIÓN DEL MANUAL

Este esquema técnico garantiza que MedReminder AI funcione como un entorno riguroso, eficiente e idóneo para la validación expedita de software médico a nivel prototipo.